

CSE 141L Milestone 1

Jonathan Osorio, A17953638

Jacob Lam, A18519912

Academic Integrity

Your work will not be graded unless the signatures of all members of the group are present beneath the honor code.

To uphold academic integrity, students shall:

- Complete and submit academic work that is their own and that is an honest and fair representation of their knowledge and abilities at the time of submission.
- Know and follow the standards of CSE 141L and UCSD.

Please sign (type) your name(s) below the following statement:

I pledge to be fair to my classmates and instructors by completing all of my academic work with integrity. This means that I will respect the standards set by the instructor and institution, be responsible for the consequences of my choices, honestly represent my knowledge and abilities, and be a community member that others can trust to do the right thing even when no one is watching. I will always put learning before grades, and integrity before performance. I pledge to excel with integrity.

Jonathan Osorio

Jacob Lam

0. Team

Jonathan Osorio

Jacob Lam

1. Introduction

Our architecture is called ARIA (Accumulator-Register ISA Architecture). The overall design philosophy is to maximize instruction encoding efficiency within a strict 9-bit instruction width while retaining enough register flexibility to handle the nested-loop array processing required by all three target programs.

ARIA is best classified as a hybrid accumulator/load-store machine. Like a classic accumulator machine, all ALU operations implicitly write their result to a single fixed register (R0), requiring only one explicit register field per instruction and freeing up bits for a wider opcode space. Like a load-store machine, memory is only accessed through explicit load and store instructions — all computation happens exclusively in registers, and memory addresses are always register-indirect with a small offset.

Our primary goals were: first, to fit a complete and practical instruction set within 9 bits without sacrificing the operations needed for XOR-based Hamming distance, multi-byte subtraction for arithmetic distance, and shift-and-add multiplication; second, to support 8 general-purpose registers so that loop counters, running min/max values, and operands can all remain in registers across the inner loop body without spilling to memory; and third, to keep the hardware implementation

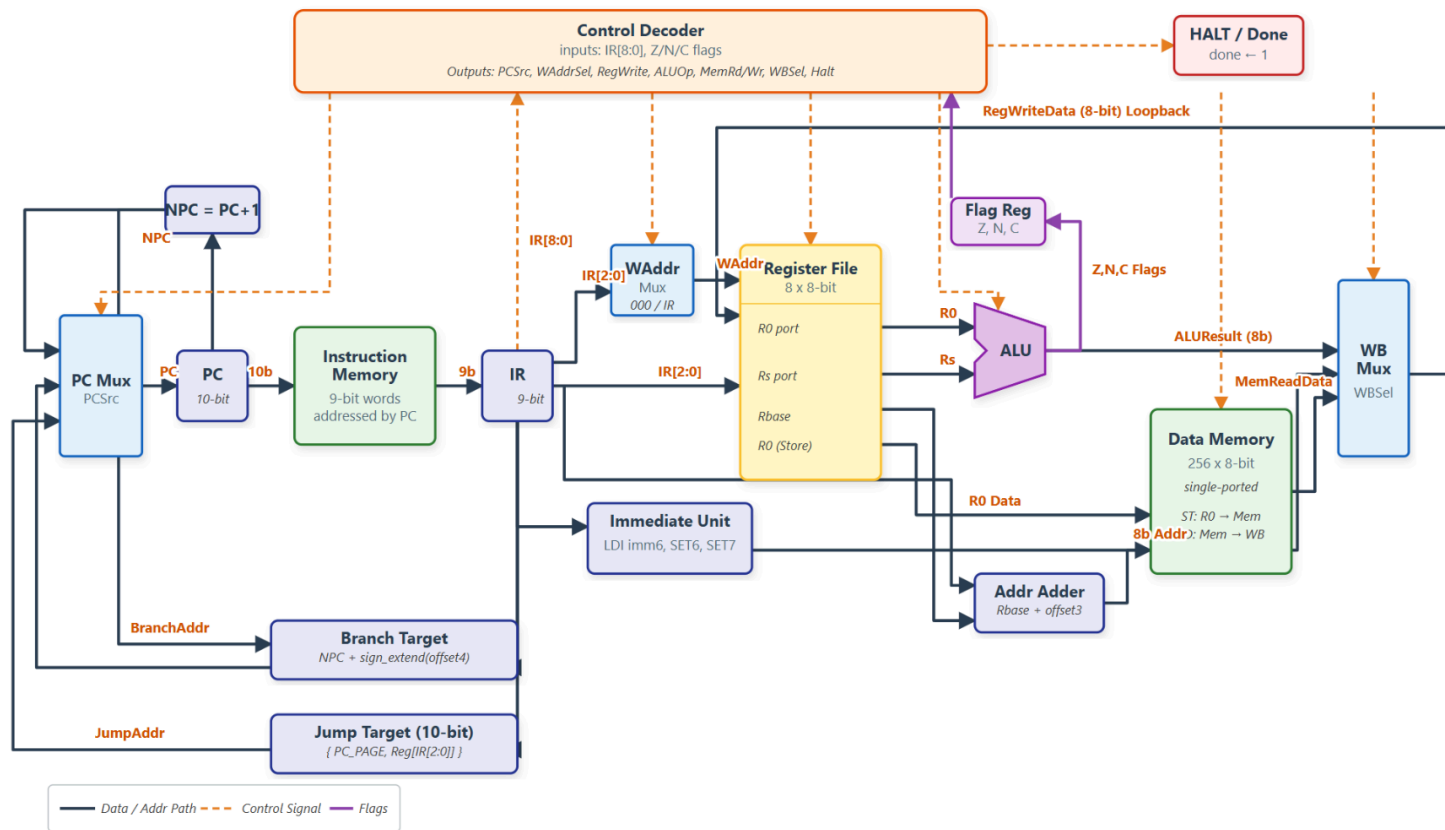
simple and regular, with a clean 2-bit format selector driving four distinct instruction formats and one implicit destination rule governing all ALU operations

2. Architectural Overview

The diagram below shows the major hardware blocks of the ARIA processor and how they connect. The processor uses a Harvard architecture with separate instruction and data memories, satisfying the project requirement for independent instruction and data memory spaces.

Accumulator/Load-Store CPU Architecture

RTL-Derived Datapath • 9-bit Instruction • 8-bit Datapath • Single-Ported Memory



The major blocks are:

- Instruction Memory: A 9-bit wide ROM holding up to 1,024 assembled instructions. Loaded via \$readmemb. Separate from data memory (Harvard).
- PC / Fetch Unit: A 10-bit program counter that increments each cycle, jumps on branches, and resets on start. Reads one 9-bit instruction per cycle from instruction memory.
- Control Decoder: Decodes the 9-bit instruction into control signals for every other block. Reads the top 2 bits (format selector), then format-specific fields for ALU op, register addresses, memory direction, branch condition, and done.
- Register File: Eight 8-bit registers (R0–R7). R0 is the accumulator — all ALU results write here. R1–R7 are general storage. The project requires at most one register write per instruction; this is satisfied by the accumulator design.
- ALU: Combinational logic performing all 16 Format A operations on R0 and one source register. Outputs an 8-bit result and three status flags: Z (zero), N (negative), C (carry).
- Flag Register: Holds Z, N, C flags from the last ALU operation. Read by the branch unit in the control decoder to determine whether to take a branch.
- Data Memory: A 256-byte single-ported RAM. The testbench writes input values into addresses 0–63 before start falls. Programs write results into addresses 64+ before asserting done via HALT.

- PC_PAGE Register: A 2-bit register set by the SETP instruction. Combined with an 8-bit register value by JR to form a 10-bit jump target, enabling jumps across the full 1,024-entry instruction space.

3. Machine Specification

ALU Operations

All ALU operations use this format:

00 0000 rrr

R0 = accumulator

Rs = register specified by rrr

Name	Type	Bit Breakdown	Example	Notes
add = add	ALU	00 0000 rrr, where 0000=0000, rrr=X	ADD R2 → 00 0000 010 → 000000010	$R0 \leftarrow R0 + R2$; updates Z, N, C
addc = add with carry	ALU	00 0000 rrr, where 0000=0001, rrr=X	ADDC R5 → 00 0001 101 → 000001101	$R0 \leftarrow R0 + R5 + C$; for multi-byte addition
sub = subtract	ALU	00 0000 rrr, where 0000=0010, rrr=X	SUB R3 → 00 0010 011 → 000010011	$R0 \leftarrow R0 - R3$; updates carry/borrow
subb = subtract with borrow	ALU	00 0000 rrr, where 0000=0011, rrr=X	SUBB R4 → 00 0011 100 → 000011100	$R0 \leftarrow R0 - R4 - C$; for multi-byte subtraction
and = bitwise AND	ALU	00 0000 rrr, where 0000=0100, rrr=X	AND R1 → 00 0100 001 → 000100001	Useful for masks and bit tests

or = bitwise OR	ALU	00 0000 rrr, where oooo=0101, rrr=X	OR R1 → 00 0101 001 → 000101001	Useful for combining bits
xor = bitwise XOR	ALU	00 0000 rrr, where oooo=0110, rrr=X	XOR R5 → 00 0110 101 → 000110101	Essential for Hamming distance
cmp = compare	ALU	00 0000 rrr, where oooo=0111, rrr=X	CMP R6 → 00 0111 110 → 000111110	Computes flags for R0 - R6, but does not overwrite R0
lsl = logical shift left	ALU	00 0000 rrr, where oooo=1000, rrr=000	LSL → 00 1000 000 → 001000000	R0 ← R0 << 1; old bit 7 goes to C
lsr = logical shift right	ALU	00 0000 rrr, where oooo=1001, rrr=000	LSR → 00 1001 000 → 001001000	R0 ← R0 >> 1; old bit 0 goes to C
rol = rotate left through carry	ALU	00 0000 rrr, where oooo=1010, rrr=000	ROL → 00 1010 000 → 001010000	Useful for multi-byte shifts
ror = rotate right through carry	ALU	00 0000 rrr, where oooo=1011, rrr=000	ROR → 00 1011 000 → 001011000	Useful for multi-byte shifts
not = bitwise NOT	ALU	00 0000 rrr, where oooo=1100, rrr=000	NOT → 00 1100 000 → 001100000	R0 ← ~R0
neg = two's-complement negate	ALU	00 0000 rrr, where oooo=1101, rrr=000	NEG → 00 1101 000 → 001101000	R0 ← -R0
clr = clear accumulator	ALU	00 0000 rrr, where oooo=1110, rrr=000	CLR → 00 1110 000 → 001110000	R0 ← 0

get = copy register into accumulator	ALU	00 oooo rrr, where oooo=1111, rrr=X	GET R3 $\rightarrow$ 00 1111 011 $\rightarrow$ 001111011	$R0 \leftarrow R3$
--------------------------------------	-----	-------------------------------------	--	--------------------

Memory Operations

Memory format:

01 m bbb qqq

Address calculation:

$\text{effective_address} = R_{\text{bbb}} + \text{qqq}$

qqq is an unsigned 3-bit offset from 0-7.

Name	Type	Bit Breakdown	Example	Notes
ld = load byte from data memory	Memory	01 m bbb qqq, where m=1, bbb=X, qqq=X	LD R3, 1 $\rightarrow$ 01 1 011 001 $\rightarrow$ 011011001	$R0 \leftarrow \text{mem}[R3 + 1]$
st = store byte to data memory	Memory	01 m bbb qqq, where m=0, bbb=X, qqq=X	ST R6, 4 $\rightarrow$ 01 0 110 100 $\rightarrow$ 010110100	$\text{mem}[R6 + 4] \leftarrow R0$

Branch Operations

Branch format:

10 ccc dddd

Target calculation:

if condition true:

$$PC \leftarrow PC + 1 + \text{sign_extend}(\text{dddd})$$

else:

$$PC \leftarrow PC + 1$$

The 4-bit signed offset supports:

-8 to +7 instructions from PC+1

Name	Type	Bit Breakdown	Example	Notes
bz = branch if zero	Branch	10 ccc dddd, where ccc=000, dddd=X	BZ +3 → 10 000 0011 → 100000011	Branch if Z=1
bnz = branch if not zero	Branch	10 ccc dddd, where ccc=001, dddd=X	BNZ -4 → 10 001 1100 → 100011100	Branch if Z=0
bn = branch if negative	Branch	10 ccc dddd, where ccc=010, dddd=X	BN -2 → 10 010 1110 → 100101110	Branch if N=1
bnn = branch if not negative	Branch	10 ccc dddd, where ccc=011, dddd=X	BNN +2 → 10 011 0010 → 100110010	Branch if N=0
bc = branch if carry/borrow	Branch	10 ccc dddd, where ccc=100, dddd=X	BC +1 → 10 100 0001 → 101000001	Branch if C=1
bnc = branch if no carry/borrow	Branch	10 ccc dddd, where ccc=101, dddd=X	BNC +1 → 10 101 0001 → 101010001	Branch if C=0
bra = branch always	Branch	10 ccc dddd, where ccc=110, dddd=X	BRA -7 → 10 110 1001 → 101101001	Unconditional short branch

bnv = branch never	Branch	10 ccc dddd, where ccc=111, dddd=X	BNV +0 → 10 111 0000 → 101110000	Useful as a branch-format NOP
--------------------	--------	------------------------------------	----------------------------------	-------------------------------

Immediate Operations

Immediate format:

110 iiiiii

Name	Type	Bit Breakdown	Example	Notes
ldi = load 6-bit immediate	Immediate	110 iiiiii, where iiiiii=X	LDI 37 → 110 100101 → 110100101	R0 ← zero_extend(37)

This only loads 0–63, must use SET6 SET7 to build larger values.

Special Operations

Special format:

111 sss rrr

Name	Type	Bit Breakdown	Example	Notes
put = copy accumulator into register	Special	111 sss rrr, where sss=000, rrr=X	PUT R4 → 111 000 100 → 111000100	R4 ← R0
jr = jump register	Special	111 sss rrr, where sss=001, rrr=X	JR R5 → 111 001 101 → 111001101	PC ← {PC_PAGE, R5}

halt = stop and assert done	Special	111 sss rrr, where sss=010, rrr=000	HALT → 111 010 000 → 111010000	Asserts done=1
nop = no operation	Special	111 sss rrr, where sss=011, rrr=000	NOP → 111 011 000 → 111011000	Does nothing
set6 = set bit 6 of accumulator	Special	111 sss rrr, where sss=100, rrr=000	SET6 → 111 100 000 → 111100000	$R0 \leftarrow R0 \mid 0x40$
set7 = set bit 7 of accumulator	Special	111 sss rrr, where sss=101, rrr=000	SET7 → 111 101 000 → 111101000	$R0 \leftarrow R0 \mid 0x80$
setp = set PC page	Special	111 sss rrr, where sss=110, rrr=X	SETP 2 → 111 110 010 → 111110010	$PC_PAGE \leftarrow rrr[1:0]$
clc = clear carry flag	Special	111 sss rrr, where sss=111, rrr=000	CLC → 111 111 000 → 111111000	$C \leftarrow 0$

Internal Operands

TODO. How many registers are supported? Is there anything special about any of the registers (e.g. constant, accumulator), or all of them general purpose?

Registers

The processor supports:

8 registers, R0–R7, Each register is 8 bits.

Register	Role
R0	Accumulator
R1	General register, often loop counter
R2	General register, often inner loop counter
R3	General register, often input pointer
R4	General register, temporary byte/value
R5	General register, temporary byte/value
R6	General register, often min/max or output pointer
R7	General register, often output pointer/scratch

Flags

The processor has three main flags:

Flag	Meaning
Z	Result was zero
N	Result was negative, meaning bit 7 of result is 1
C	Carry for addition, borrow for subtraction, shifted-out bit for shifts

Branches use these flags

The processor supports 8 registers, each 8 bits wide, labeled R0 through R7. R0 is special: it is the accumulator and the implicit destination for all Format A (ALU) instructions. R1–R7 are general-purpose storage registers with no hardware-imposed special behavior.

The processor also maintains three status flags updated by ALU operations: Z (result equals zero), N (result bit 7 is set, indicating a negative two's-complement value), and C (carry out from addition, borrow from subtraction, or shifted-out bit from shifts and rotates).

Control Flow (branches)

Eight branch conditions are supported using the Format C instruction (10 | ccc | dddd):

cond [6:4]	Mnemonic	Condition	Use case
000	BZ	Z = 1 (result was zero)	Loop termination, equality check
001	BNZ	Z = 0 (result was not zero)	Inner loop control

010	BN	N = 1 (result was negative)	Detect negative difference (absolute value)
011	BNN	N = 0 (result was non-negative)	General comparison
100	BC	C = 1 (carry/borrow set)	Multi-byte arithmetic, popcount shifts
101	BNC	C = 0 (no carry)	Multi-byte arithmetic
110	BRA	Always	Unconditional short jump
111	BNV	Never	No-op in branch format

Branch target: $PC \leftarrow NPC + \text{sign_extend}(\text{offset4})$

Large jump: $PC \leftarrow \{PC_PAGE, \text{Reg}[\text{IR}[2:0]]\}$

Branching and Jumping

In this architecture, control flow is divided into two methods based on distance: Short Conditional Branches for tight loops, and Long Register Jumps for moving anywhere else in the 1024-instruction memory space.

Standard conditional branches (like BZ and BNZ) use a 4-bit signed offset.

- Calculation: $\text{target} = \text{PC} + 1 + \text{sign_extend}(\text{offset4})$
- Range: -8 to +7 instructions from the instruction immediately following the branch.

Unconditional Long Jumps

To jump anywhere in the 1024-instruction memory limit, construct a full 10-bit address using an absolute jump (JR). Because instruction widths are small, you cannot simply write JUMP 300. You must construct the address piece by piece: load the lower bits into a register, set the upper "page" bits, and jump.

Short distance, conditional: Use standard BZ/BNZ directly.

Long distance, unconditional: Build the 10-bit address using LDI, PUT, SETP, and JR.

Long distance, conditional: Build the address, use a short branch to conditionally skip over it, and execute the JR.

Addressing Modes

The only memory addressing mode is base-register plus 3-bit unsigned offset:

$$\text{effective_address} = \text{R_base} + \text{offset} \quad (8\text{-bit result, } 0\text{--}255)$$

The base register (bbb field) holds a data memory address. The offset (qqq field, 0–7) is added unsigned. R0 is the implicit data register for both loads and stores.

Examples:

```
; Load high and low bytes of a 16-bit value at address in R3:
LD R3, 0      ; R0 <- mem[R3 + 0] (high byte)
PUT R4        ; R4 = high byte
LD R3, 1      ; R0 <- mem[R3 + 1] (low byte)
PUT R5        ; R5 = low byte
```

```
; Store a 32-bit product at address in R7:
GET R1        ; R0 = byte 3 (MSB)
ST R7, 0      ; mem[R7 + 0] = MSB
GET R2
ST R7, 1
GET R3
ST R7, 2
GET R4        ; R0 = byte 0 (LSB)
ST R7, 3      ; mem[R7 + 3] = LSB
```

Building addresses above 63 (required for output regions at addresses 64–127):

```
LDI 0         ; R0 = 0
```



```
SET6      ; R0 = 0 | 0x40 = 64
PUT R6    ; R6 = 64 (base of output region)
```

4. Programmer's Model [Lite]

The programmer should think of ARIA as a small accumulator machine with helper registers and clean load / store memory access.

The mental model is: R0 is the workbench, R1-R7 are shelves, and data memory is long-term storage.

The general strategy is as follows. Use LDI, SET6, and SET7 to construct constants and addresses at program startup, then park them in R1-R7 using PUT. Load pairs of bytes from memory into R0, immediately moving them to shelf registers before loading the next byte. Perform all arithmetic and logic through R0, for 16-bit values, process the low byte first with ADD/SUB, then the high byte with ADDC/SUBB to propagate carry. For multi-byte shifts, use ROR/ROL through carry across successive 8-bit chunks. Use CMP followed by a branch for min/max decisions. For loops, preload the loop target address into a register at startup and use the inverse-branch + JR idiom for any loop body that may exceed 8 instructions. Store final answers into data memory before executing HALT, ensuring done is asserted only after all results are written.

We cannot directly copy MIPS or ARM instructions. Both architectures assume 32-bit instruction widths and can afford to name two or three register operands per instruction. With 8 registers requiring 3 bits each, three register fields would cost 9 bits — leaving zero bits for the opcode. Even two explicit register fields leave only 3 bits for opcodes, giving just 8 operations — insufficient for our needs.

We adapt the high-level ideas: load / store memory discipline, condition-flag-based branches, and ALU operations like XOR, shifts, and add-with-carry. The key departure is making the destination implicit (always R0), which is the single compromise that makes the 9-bit budget work without sacrificing opcode coverage.

4.3 "Will your ALU be used for non-arithmetic instructions (e.g., MIPS or ARM-like memory address pointer calculations, PC relative branch computations, etc.)? If so, how does that complicate your design?"

Yes, the ALU performs non-arithmetic operations, but no, it is not used for address calculations or branch computations.

What the ALU DOES handle:

- Logical Operations: AND, OR, XOR, NOT - used for bit manipulation (e.g., XOR for Hamming distance in Program 1)
- Shift/Rotate Operations: LSL, LSR, ROL, ROR - critical for:
 - Multi-byte arithmetic (ROL/ROR through carry in Program 3's 32-bit shifts)
 - Address doubling (LSL for multiplying by 2)
 - Bit extraction in multiplication
- Data Movement: GET (copy register to R0), CLR (zero R0)

5. Program Implementation

Program 1 Pseudocode

```
min_dist = 16
max_dist = 0

for j = 0 to 31:
    for k = j + 1 to 31:
        A = load16(2 * j)
        B = load16(2 * k)

        xor_value = A XOR B
        dist = popcount16(xor_value)

        if dist < min_dist:
            min_dist = dist

        if dist > max_dist:
            max_dist = dist

mem[64] = min_dist
mem[65] = max_dist
halt

popcount16(x):

    count = 0

    loop_counter = 16

    while loop_counter != 0:

        if x[0] == 1:
```

```
        count += 1

    x = x >> 1

    loop_counter -= 1

return count
```

Program 1 Assembly Code

```
; --- Program 1 startup: build constants and pointers ---

LDI  16          ; R0 = 16

PUT  R6          ; R6 = min_dist = 16 (max possible)

CLR                     ; R0 = 0

PUT  R7          ; R7 = max_dist = 0

CLR

PUT  R1          ; R1 = i = 0


; outer loop: for i = 0 to 31

outer:

    GET  R1

    PUT  R2          ; R2 = j = i

    LDI  1
```

```

    ADD    R2

    PUT    R2          ; R2 = j = i+1


; inner loop: for j = i+1 to 31
inner:

; load A = mem[2*i], mem[2*i+1]

    GET    R1

    LSL

    PUT    R3          ; R3 = 2*i

    LD     R3, 0       ; R0 = A_high

    PUT    R4

    LD     R3, 1       ; R0 = A_low

    PUT    R5

; load B = mem[2*j], mem[2*j+1]

    GET    R2

    LSL

    PUT    R3          ; R3 = 2*j

```

```

LD    R3, 0      ; R0 = B_high

XOR   R4         ; R0 = A_high XOR B_high

PUT   R4

LD    R3, 1      ; R0 = B_low

XOR   R5         ; R0 = A_low XOR B_low

PUT   R5

; popcount on {R4, R5} -> accumulate in a counter register

    ; ... popcount loop (16 iterations, shift and count) ...

; compare to min/max

    ; ... update R6 (min) and R7 (max) using CMP + BNN/BN ...

; j++

LDI   1

ADD   R2

PUT   R2

LDI   32

CMP   R2

BNZ   inner      ; branch if j != 32

; i++

```

```
LDI  1

ADD  R1

PUT  R1

LDI  32

CMP  R1

BNZ  outer    ; branch if i != 32

; store results

LDI  0

SET6          ; R0 = 64

PUT  R3       ; R3 = 64

GET  R6

ST   R3, 0    ; mem[64] = min_dist

GET  R7

ST   R3, 1    ; mem[65] = max_dist

HALT
```

Program 2 Pseudocode

```
min_dist = 0xFFFF
max_dist = 0
for j = 0 to 31:
    for k = j + 1 to 31:
        A = signed_load16(2 * j)
        B = signed_load16(2 * k)
        diff = sign_extend_17(A) - sign_extend_17(B)
        if diff < 0:
            diff = -diff
        dist = lower16(diff)
        if dist < min_dist:
            min_dist = dist
        if dist > max_dist:
            max_dist = dist
store16(66, min_dist)
store16(68, max_dist)
halt
```

Program 2 Assembly Code


```

; 16-bit subtraction: {R4,R5} - {R6,R7} -> {R4,R5}

;   GET R5 / SUB R7 / PUT R5    (low byte, sets borrow in C)

;   GET R4 / SUBB R6 / PUT R4   (high byte, uses borrow)


; Absolute value: test sign bit of high byte (R4)

;   GET R4 / LSL                (shift sign into C)

;   BNC  positive              (if C=0, already positive)

;   GET R5 / NEG / PUT R5       (negate low byte)

;   GET R4 / NEG / ADDC R0 ...   (negate high byte + carry)


; Min/max compare is same structure as Program 1

```

Program 3 Pseudocode

```

for k = 0 to 15:

    A = signed_load16(4 * k)

    B = signed_load16(4 * k + 2)

    sign = sign(A) XOR sign(B)

    if A < 0:

```

```
    A = -A

if B < 0:

    B = -B

product = 0

multiplicand = zero_extend_32(A)

multiplier = B

counter = 0

while counter != 16:

    if multiplier[0] == 1:

        product = product + multiplicand

        multiplicand = multiplicand << 1

        multiplier = multiplier >> 1

        counter += 1

if sign == 1:

    product = -product

store32(64 + 4 * k, product)

halt
```

Program 3 Assembly Code

```
; 32-bit left shift of {P3,P2,P1,P0} using ROL through carry:

;   CLR / GET P0 / ROL / PUT P0    (shift LSB, old bit7->C)

;   GET P1 / ROL / PUT P1          (C->bit0, old bit7->C)

;   GET P2 / ROL / PUT P2

;   GET P3 / ROL / PUT P3          (MSB)


; 32-bit addition {P3,P2,P1,P0} += {M3,M2,M1,M0}:

;   CLC

;   GET P0 / ADDC M0 / PUT P0

;   GET P1 / ADDC M1 / PUT P1

;   GET P2 / ADDC M2 / PUT P2

;   GET P3 / ADDC M3 / PUT P3


; Outer loop: k = 0..15, increment output pointer by 4

; Inner loop: bit = 0..15, test LSB of multiplier each time
```

4. Change the Program Implementation number from 5 to 6.

(more steps on the next page)

5. Copy, paste, and complete the section templates we provide in the following pages to your report. Omit sections that you don't intend on using from your report (e.g. lookup table if you do not have one). We want you to test two modules: the Program Counter (to see if you can perform jumps/branches) and the ALU (to verify you have implementations for your specified instructions). Your report should consist of, in order:

0. Team
1. Introduction
2. Architectural Overview
3. Machine Specification
4. Programmer's Model
5. Individual Component Specification
 - a. Top Level
 - b. Program Counter
 - c. Instruction Memory
 - d. Control Decoder
 - e. Register File
 - f. ALU
 - g. Data Memory
 - h. Lookup Tables
 - i. Muxes
 - j. (if necessary) other modules - name sections as appropriate
6. Program Implementation

7. Changelog

5. Individual Component Specification

5.a. Top Level

Module file name

TODO

Functionality Description

TODO. Write a brief description of the functionality of this module.

Schematic

TODO. Show us your schematic for the top level.

5.b. Program Counter

Module file name

TODO

Module testbench file name

TODO

Functionality Description

TODO. Write a brief description of the functionality of this module.

(Optional) Testbench Description

TODO. Describe your testbench. How does it work? What test cases does it test?

Schematic

TODO. Show us your schematic for the fetch unit.

(Optional) Timing Diagram

TODO. Show us a screenshot of the timing diagram that demonstrates all relevant functions of the fetch unit.

5.c. Instruction Memory

Module file name

TODO

Functionality Description

TODO. Write a brief description of the functionality of this module.

Schematic

TODO. Show us your schematic for the fetch unit.

5.d. Control Decoder

Module file name

TODO

Functionality Description

TODO. Write a brief description of the functionality of this module.

Schematic

TODO. Show us your schematic for the control decoder.

5.e. Register File

Module file name

TODO

Functionality Description

TODO. Write a brief description of the functionality of this module.

Schematic

TODO. Show us your schematic for the register file.

5.f. ALU (Arithmetic Logic Unit)

Module file name

TODO

Module testbench file name

TODO

Functionality Description

TODO. Write a brief description of the functionality of this module.

(Optional) Testbench Description

TODO. Describe your testbench. How does it work? What test cases does it test?

ALU Operations

TODO. What ALU operations will you be demonstrating? What instructions are they relevant to?

Schematic

TODO. Show us your schematic for the register file.

(Optional) Timing Diagram

TODO. Show us a screenshot of the timing diagram that demonstrates all relevant operations you mentioned in the ALU Operations

section.

5.g. Data Memory

Module file name

TODO

Functionality Description

TODO. Write a brief description of the functionality of this module.

Schematic

TODO. Show us your schematic for the data memory.

5.h. Lookup Tables

Module file name

TODO

Functionality Description

TODO. Write a brief description of the functionality of this module.

Schematic

TODO. Show us your schematic(s) for the lookup table(s).

5.i. Muxes (Multiplexers)

Module file name

TODO

Functionality Description

TODO. Write a brief description of the functionality of this module.

Schematic

TODO. Show us your schematic for your mux(es).

5.j. Other Modules (if necessary)

Module file name

TODO

Functionality Description

TODO. Write a brief description of the functionality of this module.

Schematic

TODO. Show us your schematic for your module.

7. Changelog

Example changes below:

- Milestone 2
 - Introduction
 - edited to change from a load/store architecture to accumulator architecture.
 - TODO: add bullet points as necessary
- Milestone 1
 - Initial version